Fast Correlation Processing

David Ohm, PhD

In this paper, we review the theory and implementation of a fast correlation method called Overlap-Scrap, also known as Overlap-Save. Sample Matlab code is provided for learning and experimentation.

Introduction

There are two methods for computing the correlation of real or complex-valued signals. The first method uses a time-domain approach, and for a stationary process can be computed using the following formula

$$r_{xh}(k) = \begin{cases} \frac{1}{N}\sum_{n=0}^{N-1} x(n+k)h^*(n), & 0 \leq k \leq N-1 \\ r^*_{xh}(-k), & -(N-1) \leq k < 0 \\ 0, & \text{elsewhere} \end{cases} \tag{1}$$

where N is the input data record length, $x(n)$ is the input signal and $h(n)$ is the kernel, or reference signal. The computational time to compute a correlation using this method is directly proportional to the number of samples in the kernel. Therefore, the longer the kernel data record the more time it takes to compute a correlation. The second method for computing a correlation utilizes a frequency-domain approach, often called "fast correlation". Fast correlation uses the Fast Fourier Transform (FFT) to transform the input signal and the kernel signal into the frequency domain, and then exploits the following mathematical relationship

$$x(n) \circledast h(n) = IFFT\{X(k) * conj\{Y(k)\}\} \tag{2}$$

It is well known that when dealing with kernel lengths that are greater than approximately 60 samples it is faster, or more efficient, to use the fast correlation method. When using the fast correlation method, the time it takes to compute the correlation is proportional to the logarithm of the number of samples, and therefore, changes slowly as the kernel length increases. Figure 1 shows a computation time comparison between the time-domain and fast correlation methods versus kernel length. A block diagram showing the processing steps involved in computing the fast correlation is shown in Figure 2. It is worth mentioning that fast correlation is also called "pulse compression" in the radar community.

Overlap "Scrap" Algorithm

The most efficient way to implement fast correlation for very long input data sequences is to use the overlap "scrap" (OVS) algorithm, also known as the overlap-save algorithm. The overlap-save algorithm can be easily understood by looking at the sample sequences x and h in Figure 3. First, the input data sequence is broken up into arbitrary segments of length N. Then, each segment is processed using the fast correlation method discussed in the last section. However, special segment overlapping and data discarding rules must be by followed in order to avoid

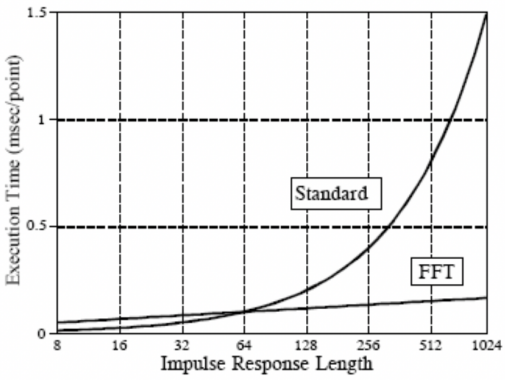

Figure 1: *Execution times for fast correlation compared to time-domain correlation. [Steven W. Smith, DSP Guide].*

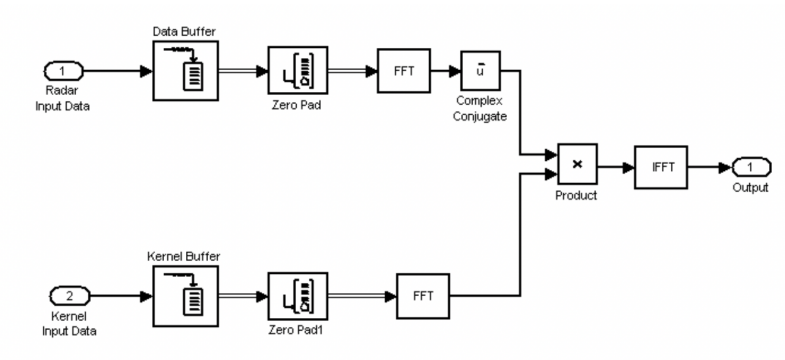

Figure 2: *Block diagram showing the processing steps for computing frequency-domain correlation.*

wrap-around pollution in the output sequence. We will not go into an analysis of wrap-around pollution in this report since most DSP textbooks cover it in detail.

The OVS algorithm can be implemented by following these steps:

1. Zero pad the kernel sequence out to the FFT length (N) to avoid wrap-around pollution
2. Add M-1 zeros to the beginning of the input sequence and N-1 zeros to the end
3. Break input signal into successive segments of N samples, each segment overlapping the previous segment by M-1 samples
4. Compute the FFT of the kernel and complex conjugate (save for reuse)
5. Buffer a segment of input signal and compute the FFT
6. Multiply (element- wise) outputs from steps 4 and 5
7. Compute the inverse FFT (IFFT) from output of multiply
8. Last M-1 samples of each IFFT output are discarded, (front M-1 samples discarded from first output block)
9. The remaining samples are used as output
10. Steps 5-9 are repeated until all input segments used

x = 0000000000001011000 (input) h = 1011 (kernel)		
Input Segment	*Output Segment*	*Output after M-1 Scrap*
x1 = 00000000	y1 = 00000000	Yout1 = 00000
x2 = 00000001	y2 = 00001101	Yout2 = 00001
x3 = 00101100	y3 = 11311101	Yout3 = 11311
x4 = 10000000	y4 = 10000110	Yout4 = 10000
Correlator Output = 00000000011131110000		
Remove front M-1 Zeros, Yout=00000011131110000		

Figure 3: *Overlap "Scrap" algorithm segmentation concept.*

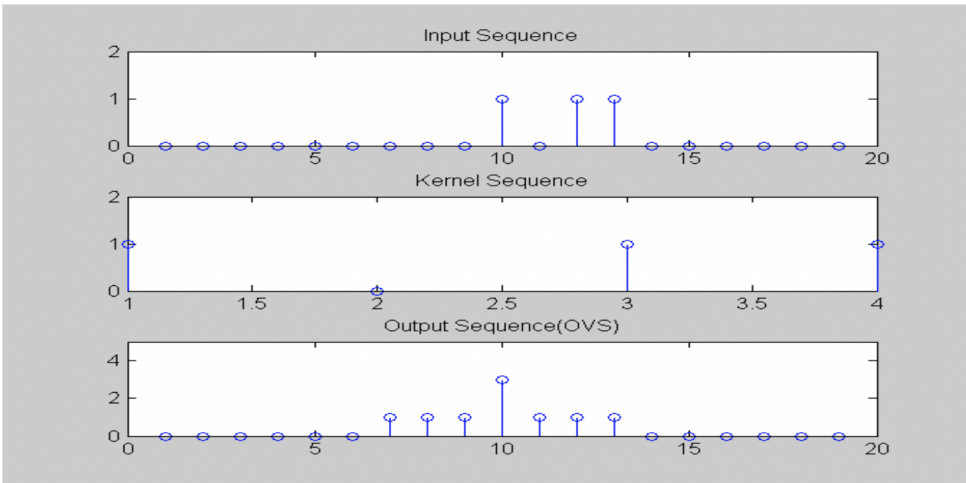

Figure 4: *Plots from simple Matlab example.*

It is important to note that the last M-1 samples of each IFFT output should be discarded instead of the front M-1 samples. This will help in the implementation by avoiding double buffering at the output of the IFFT. Matlab code for simulating the OVS correlation algorithm is included at the end of this report.

Usually we would consider the following factors when deciding the overlap amount for the OVS algorithm.

1. FFT Accuracy – the numerical accuracy of fast correlation is dependent on the error introduced by the FFT and IFFT process. For floating point implementations this is usually negligible, but for fixed point processing, significant dynamic range can be lost when using larger transforms.

2. Latency – the fast correlation process extends the group delay by at least N samples. So, the longer the FFT, the longer the latency. In the absence of a benchmark, an overlap factor of 4 to 8 is a good rule of thumb. Since we do not have a lot of flexibility when choosing a kernel size, we will need to choose the length of the FFT to give optimal performance.

Example Matlab Code

```
% Filename: overlap_scrap_corr.m
% Author: drohm
%
% y = output sequence
% x = input sequence
% h = kernel sequence
% N = segment length & fft length
% Implements overlap-scrap (OVS) "overlap-save" correlation.
%==============================================================
%==============================================================
clear all;close all
x1 = [zeros(1,128)+j*zeros(1,128)];   %Input Data
h1 = [ 1 0 1 1 0 1];                  %Kernel
Lenx = length(x1); M = length(h1);
x1(25:25+M-1)=h1;                     %Embed Kernel into nput data

xp = [x1 zeros(1,length(h1)-1)];      %Zero Padded input sequence and Kernel
hp = [h1 zeros(1,length(x1)-1)];

N=8; % Segment Size and FFT Size for OVS
M1 = M-1; L = N-M1;
h = [h1 zeros(1,N-M)];
Leny = Lenx + M -1; %Length of final correlation output

x = [zeros(1,M-1), x1, zeros(1,N-1)];
K = floor((Lenx+M1-1)/(L)); % # of segment sent to correlator
Y = zeros(K+1,N); % Allocate output 2D array of zeros

%%%%% Correlator %%%%%
% These steps perform the correlation using the FFT
for k=0:K
  xk = x(k*(N-(M-1))+1:k*(N-(M-1))+N); % Input segments
  Y(k+1,:) = ifft( (fft(xk,N).*conj(fft(h,N))),N ); % FFT Correlation
end
Y = Y(:,1:N-(M-1)) ; %discard the last (M-1) samples
y = (Y(:)) ; % assemble output into 1D array
y = (real(y(M-1:Leny))); %remove extra zeros at end

%%%%% Plotting %%%%%
figure(1)
subplot(4,1,1)
stem(x1);title( Input Data );ylim([0,2])
subplot(4,1,2)
stem(h1);title( Kernel Data );ylim([0,2])
subplot(4,1,3)
stem(y);title( Output (OVS) );ylim([0,5])
```